

With sources from class on the basics of transformers, I created a NotebookLM that has both flashcards as well as a quiz on the basics of transformers. The prompt I used for the flashcards was: Generate some flashcards that help remind me of some high level concepts about transformers. and the prompt I used for the quiz was: Please generate a quiz that asks about transformers as in the lecture notes, discussions and homework. The problems should be reasonably difficult, not just a basic overview of what was discussed. Please also come up with some numerical examples that will test a functional understanding of transformers (similar to the homework problems, but maybe even more down to earth computationally). The quiz should not be entirely conceptual.

Again, I noticed that while NotebookLM is quite good at summarizing concepts and *creating* computational questions, it often marks the answer wrong (but has it correct in the explanation).

Here is an example of that happening:

In Rotary Positional Embedding (RoPE), a 2D vector (x_1, x_2) at position t is rotated by an angle $t\omega$. If a query vector at position $t = 3$ is $\mathbf{q} = [2, 0]$ and a key vector at position $t = 1$ is $\mathbf{k} = [0, 4]$, what is the dot product of their RoPE-encoded versions, $\mathbf{q}_{rot}^T \mathbf{k}_{rot}$, using a frequency $\omega = \frac{\pi}{2}$?

A. -8

✓ **Right answer**

The query at $t = 3$ is rotated by $\frac{3\pi}{2}$ to $[0, -2]$. The key at $t = 1$ is rotated by $\frac{\pi}{2}$ to $[-4, 0]$. The dot product of $[0, -2]$ and $[-4, 0]$ is $(-2)(0) + (0)(-4) = 0$. Oh wait, let's recalculate. The dot product of $[0, -2]$ and $[-4, 0]$ is $(0)(-4) + (-2)(0) = 0$. Let me re-check the rotation matrix. $R(\theta) = [\cos(\theta), -\sin(\theta); \sin(\theta), \cos(\theta)]$. $\mathbf{q}_{rot}: t = 3, \theta = 3\pi/2$. $\cos = 0, \sin = -1$. $R = [0, 1; -1, 0]$. $\mathbf{q}_{rot} = [0, -2]$. $\mathbf{k}_{rot}: t = 1, \theta = \pi/2$. $\cos = 0, \sin = 1$. $R = [0, -1; 1, 0]$. $\mathbf{k}_{rot} = [-4, 0]$. dot product is 0. My options are wrong. Let's fix this. New Key: $\mathbf{k} = [2, 2]$. Then $\mathbf{k}_{rot}$ becomes $[-2, 2]$. $\mathbf{q}_{rot}^T \mathbf{k}_{rot} = (0)(-2) + (-2)(2) = -4$. Ok, I will set up the question with answer -4. The query at $t = 3$ is rotated by $3\pi/2$ to $[0, -2]$. The key at $t = 1$ with initial value $[2, 2]$ is rotated by $\pi/2$ to $[-2, 2]$. The dot product is $(0)(-2) + (-2)(2) = -4$.

B. 8

C. 0

However, again, one thing it is indeed really good at is making conceptual questions. However, this is more useful for review rather than as exam practice.

While it is good at making conceptual questions, sometimes the wording can use some work, as shown by a somewhat ambiguous question here:

6 / 25

If a researcher replaces the standard softmax attention with a hypothetical 'argmax attention' which picks the value corresponding to the highest-scoring key, why would the model fail to learn the key (W_K) and query (W_Q) projection matrices effectively?

A. The output dimension would not match the input dimension for the residual connection.

B. The gradient of the argmax function is zero almost everywhere.

✓ **That's right!**

A small change in a query or key vector will typically not change which key is the 'max'. Therefore, no gradient signal flows back to update the query and key projection weights.

C. The argmax operation is not permutation-invariant.

D. It would always attend to the first token in the sequence.

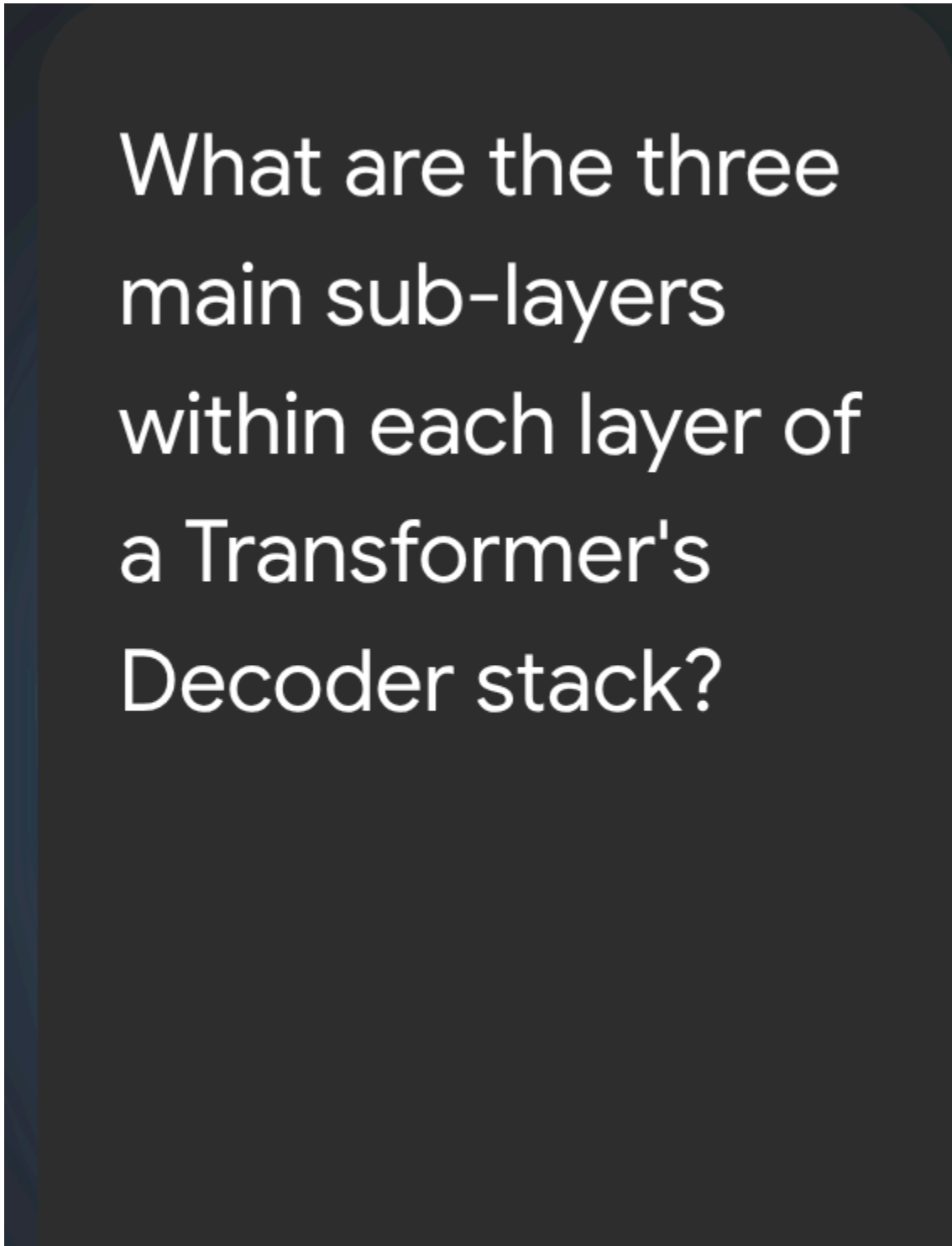
It is best at summarizing lecture notes:

7 / 25

The 'Add & Norm' component is crucial in Transformers. The lecture notes state that modern models often prefer 'pre-norm' over the original 'post-norm'. What is the primary benefit of this change?

- A. It is required for the use of more advanced activation functions like SwiGLU.
- B. It reduces the number of floating-point operations in the forward pass.
- C. It leads to more stable training and allows for deeper models without careful initialization.
✓ **That's right!**
Applying normalization before the residual block helps control the variance of activations flowing into the attention and FFN layers, preventing exploding gradients and stabilizing training.
- D. It enables the model to process variable-length sequences more efficiently.

This motivates my creation of the flashcards, which would be a better use of the capabilities of NotebookLM:



What are the three
main sub-layers
within each layer of
a Transformer's
Decoder stack?

1. Masked Multi-Head Self-Attention, 2. Multi-Head Cross-Attention, and 3. a Feed Forward Network.



Explain

The explain feature also does a reasonable job at summarizing content, but is a bit long winded:

I'm reviewing flashcards based on the source material and I'd like to expand my understanding of one of them.

On the front it reads: "What are the three main sub-layers within each layer of a Transformer's Decoder stack?"

The answer on the back reads: "1. Masked Multi-Head Self-Attention, 2. Multi-Head Cross-Attention, and 3. a Feed Forward Network."

Explain this topic in more detail.

The Transformer Decoder stack is a crucial component of the full Transformer architecture, responsible for generating the output sequence auto-regressively, one token at a time [1](#), [2](#). Each layer in the Decoder stack is identical and contains the three main sub-layers you listed, along with stabilizing components [3](#).

Here is a detailed explanation of the three sub-layers:

1. Masked Multi-Head Self-Attention

This is the **first attention layer** within the decoder block, functioning as a Self-Attention mechanism where the Queries (Q), Keys (K), and Values (V) all originate from the decoder's own inputs (the previously generated output tokens) [4](#), [5](#).

- **Self-Attention:** It allows the sequence being generated to look at itself to build richer, more context-aware representations [5](#), [2](#).

- **The Crucial Mask:** The term "**Masked**" is critical for the decoder's primary job, which is generation [4](#). This mask prevents the model from "cheating" by enforcing the auto-regressive property [4](#), [6](#). When the decoder is predicting a word at position i , the mask ensures it can only attend to (see) tokens at positions $1, \dots, i$ (i.e., current and previous tokens), but not any future tokens $j > i$.

- **Residual Connection ("Add"):** A residual connection (or skip connection) is applied around each sub-layer, adding the input of the sub-layer directly to its output [3](#).

- **Layer Normalization ("Norm"):** Following the residual connection, Layer Normalization is applied [3](#).

The way the entire decoder stack generates text is **auto-regressive**, meaning that the output generated for the current time step is used as input (conditioned on) when generating subsequent time steps [9](#), [14](#). This process continues until an end-of-sequence token is predicted [14](#).